

Design and Implementation of a Custom Deep Learning Framework

GNR638: Machine Learning for Remote Sensing II

Group 28

Team Members: 22B3932, 23B4207

Spring 2025-26

Contents

1	Introduction	4
2	Framework Architecture and Design	4
2.1	System Architecture Overview	4
2.2	Tensor Abstraction	5
2.3	Automatic Differentiation Engine	5
2.4	Mathematical Operations	6
2.5	Convolutional Layer Implementation	6
2.6	Activation and Pooling Layers	7
2.7	Fully Connected Layer	7
2.8	Loss Function	7
2.9	Optimization	8
2.10	Python-C++ Integration	8
2.11	Reproducibility	8
2.12	Memory Management Strategy	8
3	Dataset Handling and Preprocessing	9
3.1	Dataset Format	9
3.2	Dataset Implementation	9
3.3	Image Processing Pipeline	9
3.4	Tensor Conversion	10
3.5	Batching Strategy	10
3.6	Train-Validation Split	11
3.7	Dataset Loading Time Measurement	11
3.8	Memory Efficiency Considerations	11
3.9	Scalability Considerations	12

4	CNN Architecture Design and Rationale	12
4.1	Network Architecture	12
4.2	Why Convolution Layers?	13
4.3	Why ReLU Activation?	13
4.4	Why He Initialization?	13
4.5	Why MaxPooling?	14
4.6	Why Fully Connected Layers?	14
4.7	Trade-Off Analysis	14
4.8	Computational Complexity	15
5	Training Procedure and Optimization	15
5.1	Overview	15
5.2	Forward Propagation	16
5.3	Loss Function	16
5.4	Backward Propagation	16
5.5	Parameter Update Rule	17
5.6	Mini-Batch Training	17
5.7	Training Metrics	18
6	Validation and Evaluation	18
6.1	Standalone Evaluation Script	18
6.2	Dataset Loading During Evaluation	18
6.3	Model Reconstruction	19
6.4	Efficiency Metrics During Evaluation	19
6.5	Evaluation Loop	19
6.6	Memory Considerations During Evaluation	19
6.7	Experimental Results on Dataset 1 (MNIST)	20
6.7.1	Dataset Statistics	20
6.7.2	Model Complexity	20
6.7.3	Training Loss Curve	22
6.7.4	Accuracy Curves	22
6.7.5	Epoch Time Analysis	23
6.7.6	Metrics	23
6.7.7	Model Complexity vs Performance	23
6.8	Experimental Results on Dataset 2 (CIFAR-100)	24
6.8.1	Dataset Statistics	24
6.8.2	Model Complexity	24
6.8.3	Training Loss Curve	24
6.8.4	Accuracy Curves	25
6.8.5	Epoch Time Analysis	25
6.8.6	Model Complexity vs Performance	26
6.9	Data Loading Efficiency	26

7	Failed Design Decisions and Problems Faced	27
7.1	Memory Efficiency	27
7.2	Improper Weight Initialization	28
7.3	Incorrect Bias Broadcasting	28
7.4	Missing <code>parameters()</code> Implementations	29
7.5	Development Environment	29
8	Conclusion	29

1 Introduction

This assignment focuses on the design and implementation of a custom deep learning framework from first principles. The objective is to develop all core neural network components — including tensor abstractions, automatic differentiation, convolutional layers, and optimization algorithms — without relying on any existing deep learning or numerical computing libraries.

The framework must support image classification using a Convolutional Neural Network (CNN) trained on the provided datasets. All computations are implemented from scratch, and no external deep learning frameworks (such as PyTorch, TensorFlow, or NumPy) are used.

To ensure computational efficiency, the backend is implemented in C++, while Python 3.12 serves as the frontend interface using pybind11 bindings. This hybrid design enables performance optimization while maintaining ease of experimentation.

2 Framework Architecture and Design

The framework supports:

- Reverse-mode automatic differentiation
- Convolutional and fully connected layers
- Activation and pooling operations
- Cross-entropy loss
- Stochastic Gradient Descent (SGD)

2.1 System Architecture Overview

The framework follows a modular layered architecture:

1. **Core Tensor Engine (C++)**
2. **Autograd Engine (C++)**
3. **Neural Network Layers (C++)**
4. **Optimization Module (C++)**
5. **Loss Functions (C++)**
6. **Python Frontend Interface**

Each component is described in detail below.

2.2 Tensor Abstraction

The **Tensor** class forms the foundational data structure of the framework. It encapsulates:

- **data**: Contiguous storage of floating-point values
- **grad**: Gradient buffer of identical size
- **shape**: Dimensionality metadata
- **requires_grad**: Boolean flag for gradient tracking
- **parents**: Pointers to predecessor tensors in computation graph
- **backward_fn**: Local backward function

All tensor data is stored in contiguous memory using `std::vector<float>`, ensuring:

- Cache-friendly memory access
- Low memory fragmentation
- Simplified indexing arithmetic

The tensor supports arbitrary dimensional shapes but internally operates using flattened indexing for computational efficiency.

2.3 Automatic Differentiation Engine

Reverse-mode automatic differentiation (backpropagation) is implemented manually. Each operation:

1. Performs forward computation
2. Constructs a result tensor
3. Stores references to parent tensors
4. Defines a lambda-based **backward_fn**

During backward propagation:

- A topological order of the computation graph is constructed
- Gradients are propagated in reverse order
- Accumulation is performed directly into parent gradient buffers

The backward mechanism follows the computational graph approach:

$$\text{Loss} \rightarrow \text{Output} \rightarrow \cdots \rightarrow \text{Parameters}$$

Care is taken to:

- Avoid gradient overwriting
- Properly handle broadcasted operations
- Ensure memory is released after batch completion

2.4 Mathematical Operations

The following operations are implemented manually:

- Element-wise addition (with bias broadcasting)
- Element-wise multiplication
- Matrix multiplication
- Summation

Matrix multiplication uses triple nested loops:

$$C_{ij} = \sum_k A_{ik} B_{kj}$$

The backward pass computes:

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial C} B^T$$

$$\frac{\partial L}{\partial B} = A^T \frac{\partial L}{\partial C}$$

All gradients are accumulated explicitly using nested loops.

2.5 Convolutional Layer Implementation

The convolutional layer is implemented using explicit nested loops over:

- Batch dimension
- Output channels
- Spatial height and width
- Input channels
- Kernel height and width

The convolution is computed as:

$$Y = W * X + b$$

where each output activation is computed via multiply-accumulate operations over the receptive field.

Padding and stride are handled explicitly through index arithmetic.

Weights are initialized using He initialization:

$$W \sim \mathcal{U}(-\sqrt{\frac{2}{fan_in}}, \sqrt{\frac{2}{fan_in}})$$

which ensures variance preservation across ReLU layers.

2.6 Activation and Pooling Layers

ReLU activation is implemented as:

$$f(x) = \max(0, x)$$

Backward propagation masks negative inputs.

Max pooling reduces spatial dimensions by selecting maximum values within non-overlapping windows. Gradient propagation routes gradients only to the maximum element within each window.

2.7 Fully Connected Layer

The linear layer performs:

$$Y = XW + b$$

Bias broadcasting is handled explicitly.

Weights are also initialized using He initialization to maintain stable gradient flow.

2.8 Loss Function

Cross-entropy loss with numerically stable softmax is implemented as:

$$\text{Softmax}(z_i) = \frac{e^{z_i - \max(z)}}{\sum_j e^{z_j - \max(z)}}$$

Loss:

$$L = -\frac{1}{N} \sum_{i=1}^N \log p_{y_i}$$

Gradients are computed analytically as:

$$\frac{\partial L}{\partial z} = \frac{1}{N}(p - y)$$

2.9 Optimization

Stochastic Gradient Descent (SGD) is implemented as:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

Gradient buffers are reset using `zero_grad()` before each backward pass.

2.10 Python–C++ Integration

The backend is exposed to Python using `pybind11`. The Python frontend:

- Handles dataset loading
- Constructs the model
- Executes training loop
- Computes metrics
- Saves and loads model weights

This design achieves:

- Performance-critical computation in C++
- Flexibility in experimentation via Python

2.11 Reproducibility

Reproducibility is ensured by:

- Controlling random seed in both C++ and Python
- Removing time-based random initialization
- Deterministic data shuffling and JSON-based model weight saving

The backend exposes:

```
mf.set_seed(seed)
```

This ensures consistent weight initialization and training behavior across runs.

2.12 Memory Management Strategy

To prevent memory explosion during training:

- Intermediate tensors are deleted after each batch
- Python garbage collection is invoked
- No global computation graphs are stored

This ensures training completes within memory limits for all epochs.

3 Dataset Handling and Preprocessing

3.1 Dataset Format

The provided datasets follow an *image-folder structure*. The parent directory contains multiple subdirectories, where each subdirectory corresponds to a semantic class. Each class directory contains PNG images.

The directory structure is:

```
dataset/  
  class_0/  
    img1.png  
    img2.png  
    ...  
  class_1/  
    img1.png  
    ...  
  ...
```

Class labels are inferred directly from subdirectory names. A mapping dictionary `class_to_idx` is constructed during dataset initialization.

3.2 Dataset Implementation

A custom `ImageFolderDataset` class is implemented in Python. The dataset class performs:

1. Directory traversal using Python's `os` module
2. Automatic label assignment from folder names
3. Image reading using OpenCV (`cv2.imread`)
4. Conversion to 32×32 tensors
5. Conversion to framework Tensor objects

Only OpenCV is used for image loading, strictly limited to disk reading and resizing operations.

3.3 Image Processing Pipeline

For each image:

1. The image is read using:

`cv2.imread(path)`
2. If the dataset is grayscale, it is loaded as single-channel.

3. The image is resized to:

$$32 \times 32$$

4. Pixel values are converted to floating point and normalized to:

$$[0, 1]$$

5. The image is converted into shape:

$$(C, H, W)$$

Normalization is performed using:

$$x_{normalized} = \frac{x}{255.0}$$

No external numerical libraries (e.g., NumPy) are used for tensor computation.

3.4 Tensor Conversion

After preprocessing, image data is flattened and passed into the custom C++ Tensor constructor:

```
Tensor(data, shape, requires_grad=False)
```

The shape for each sample is:

$$[Channels, 32, 32]$$

Batch tensors are constructed manually by concatenating flattened image data and reshaping into:

$$[Batch_Size, Channels, 32, 32]$$

This explicit construction ensures full control over memory layout.

3.5 Batching Strategy

Mini-batch training is implemented manually.

For a given batch:

1. Indices are sampled
2. Images and labels are retrieved
3. Flattened data is aggregated
4. A single Tensor is created for the batch

The number of batches per epoch is computed as:

$$\text{num_batches} = \left\lceil \frac{N}{B} \right\rceil$$

where:

- N = total number of training samples
- B = batch size

This batching approach avoids storing full dataset tensors in CPU memory simultaneously, improving memory efficiency.

3.6 Train-Validation Split

The dataset is split into training and validation subsets using random shuffling.

Given validation split ratio α :

$$\text{split index} = N \times (1 - \alpha)$$

Random seed control ensures deterministic splits.

Validation is performed at the end of each epoch.

3.7 Dataset Loading Time Measurement

Dataset loading time is measured as:

$$T_{load} = t_{after_init} - t_{before_init}$$

This value is printed during:

- Training execution
- Evaluation script execution

This ensures transparency of I/O overhead.

3.8 Memory Efficiency Considerations

To ensure memory efficiency:

- Images are not stored as large tensors permanently
- Batch tensors are deleted after each iteration
- Python garbage collection is invoked
- No unnecessary tensor copies are created

The dataset loader stores only file paths and label indices, minimizing memory footprint.

3.9 Scalability Considerations

The implementation scales linearly with dataset size:

$$O(N)$$

since each image is read exactly once per epoch.

Because images are processed on demand rather than preloaded entirely into memory, the framework can handle larger datasets without exceeding memory limits.

4 CNN Architecture Design and Rationale

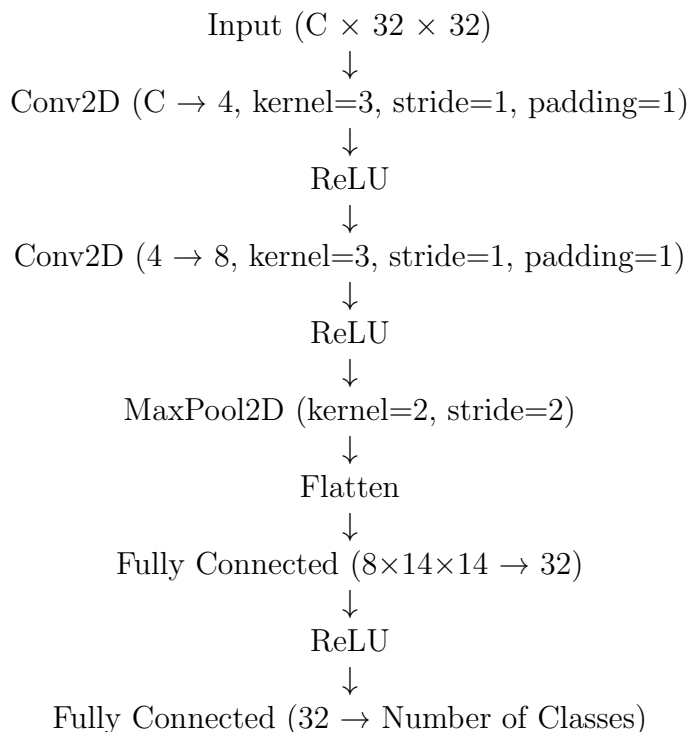
The objective was to design a convolutional neural network that:

- Satisfies assignment constraints (at least one Conv, Pooling, Activation, FC layer)
- Remains computationally efficient
- Maintains reasonable representational capacity
- Completes training within the required time constraints

The architecture was intentionally kept lightweight to balance model complexity and performance.

4.1 Network Architecture

The final architecture is:



4.2 Why Convolution Layers?

Convolution layers were used because:

- They exploit spatial locality
- They reduce parameter count compared to fully connected layers
- They enable translation invariance

Parameter sharing significantly reduces complexity:

$$\text{Conv Params} = C_{out} \times C_{in} \times K^2$$

Compared to fully connected layers:

$$\text{FC Params} = N_{in} \times N_{out}$$

Thus, convolutions are computationally efficient for image data.

4.3 Why ReLU Activation?

ReLU was chosen because:

- Avoids vanishing gradient problem
- Computationally cheap
- Works well with He initialization

Mathematically:

$$\text{ReLU}(x) = \max(0, x)$$

Derivative:

$$\frac{d}{dx}\text{ReLU}(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

This enables sparse activation and stable gradient flow.

4.4 Why He Initialization?

Since ReLU was used, He initialization was chosen:

$$W \sim \mathcal{U}\left(-\sqrt{\frac{2}{fan_in}}, \sqrt{\frac{2}{fan_in}}\right)$$

This ensures variance preservation across layers:

$$Var(Wx) \approx Var(x)$$

Without proper initialization, activations either explode or vanish.

Empirically, switching from uniform random initialization to He initialization significantly improved convergence stability.

4.5 Why MaxPooling?

MaxPooling was used to:

- Reduce spatial resolution
- Decrease computation
- Introduce translation invariance

Pooling reduces computation by factor:

$$\frac{H \times W}{(H/2) \times (W/2)} = 4$$

This reduces downstream fully connected parameters dramatically.

4.6 Why Fully Connected Layers?

Fully connected layers serve as:

- Feature aggregators
- Class decision boundaries

The first FC layer reduces 1568 features to 32, performing dimensionality compression.
The final FC layer maps:

$$32 \rightarrow \text{Number of Classes}$$

4.7 Trade-Off Analysis

The model balances:

- Depth (2 convolution layers)
- Capacity (32 hidden neurons)
- Computational efficiency

Increasing depth or channel count significantly increases:

$$\text{Parameters} \propto C_{out} \times C_{in}$$

Thus, a deeper architecture was avoided to ensure:

- Training under 3 hours
- Low memory footprint
- Fast evaluation

4.8 Computational Complexity

Forward pass complexity:

$$O(B \times C_{out} \times C_{in} \times K^2 \times H \times W)$$

This scales linearly with batch size.

Memory complexity:

$$O(\text{Parameters} + \text{Activations})$$

Since activations are cleared per batch, memory remains bounded.

5 Training Procedure and Optimization

5.1 Overview

The training process consists of iterative optimization of network parameters using mini-batch gradient descent. The objective is to minimize the multi-class cross-entropy loss over the training dataset.

Each epoch consists of:

1. Shuffling the training dataset
2. Dividing it into mini-batches
3. Forward propagation
4. Loss computation
5. Backward propagation
6. Parameter update using SGD

5.2 Forward Propagation

For each mini-batch, the input tensor is propagated through the network layers:

$$x \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool} \rightarrow \text{Flatten} \rightarrow \text{FC} \rightarrow \text{ReLU} \rightarrow \text{FC}$$

The final layer produces logits:

$$\mathbf{z} \in R^{B \times C}$$

where:

- B = batch size
- C = number of classes

5.3 Loss Function

Multi-class cross-entropy loss.

For a single sample:

$$\mathcal{L} = -\log \left(\frac{e^{z_y}}{\sum_{j=1}^C e^{z_j}} \right)$$

For a mini-batch:

$$\mathcal{L}_{batch} = \frac{1}{B} \sum_{i=1}^B \mathcal{L}_i$$

Numerical Stability

To avoid overflow in exponentiation:

$$z_j \leftarrow z_j - \max(z)$$

This improves numerical stability during softmax computation.

5.4 Backward Propagation

Automatic differentiation is implemented manually using:

- Parent tracking
- Local backward functions
- Reverse topological traversal

For cross-entropy with softmax, the gradient simplifies to:

$$\frac{\partial \mathcal{L}}{\partial z_j} = \frac{1}{B}(p_j - 1_{j=y})$$

where:

- p_j is softmax probability
- $1_{j=y}$ is indicator for correct class

This avoids explicitly computing Jacobians.

5.5 Parameter Update Rule

Stochastic Gradient Descent (SGD):

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}$$

where:

- η = learning rate
- θ = model parameters

Gradient Resetting

Before each backward pass:

$$\nabla \theta \leftarrow 0$$

This prevents gradient accumulation across batches.

5.6 Mini-Batch Training

Mini-batch size B is chosen as a trade-off between:

- Memory consumption
- Gradient stability
- Training speed

Larger batches:

- Reduce gradient variance
- Increase memory usage

Smaller batches:

- Increase noise in gradients
- Potentially improve generalization

5.7 Training Metrics

For each epoch, the following metrics are computed:

- Training loss
- Training accuracy
- Validation accuracy
- Epoch time

All metrics are stored in:

`training_logs.json`

6 Validation and Evaluation

During training, a portion of the training dataset is held out as a validation set. This validation set is used to:

- Monitor model generalization
- Detect overfitting
- Tune hyperparameters

6.1 Standalone Evaluation Script

A separate evaluation script (`evaluate.py`) is implemented.

The script requires only:

- Path to test dataset directory
- Path to saved model weights

It does not require any code modification.

Command format:

```
python evaluate.py --data_path <test_dataset_path> \  
                  --weights_path model_weights.json
```

6.2 Dataset Loading During Evaluation

Dataset loading time is measured and reported:

$$\text{Dataset Loading Time} = t_{\text{after load}} - t_{\text{before load}}$$

This metric is printed during evaluation as required.

6.3 Model Reconstruction

During evaluation:

1. Model architecture is instantiated
2. Saved weights are loaded
3. Forward pass is executed on test dataset

Weights are loaded from a JSON file and assigned to each parameter tensor.

6.4 Efficiency Metrics During Evaluation

The following metrics are computed and printed:

- Total trainable parameters
- MACs per forward pass
- FLOPs per forward pass

These metrics remain constant for a fixed architecture and are recomputed for completeness.

6.5 Evaluation Loop

The evaluation loop:

1. Processes data in mini-batches
2. Performs forward pass
3. Computes classification accuracy

No backward pass is performed during evaluation.

Accuracy is computed as:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Samples}} \times 100$$

6.6 Memory Considerations During Evaluation

To prevent memory overflow:

- Intermediate tensors are explicitly deleted
- No gradient tracking is required
- No computation graph expansion occurs

This ensures evaluation completes within the 1-hour constraint.

6.7 Experimental Results on Dataset 1 (MNIST)

6.7.1 Dataset Statistics

- Total samples: 60,000
- Training samples: 48,000
- Validation samples: 12,000
- Input channels: 1 (Grayscale)
- Number of classes: 10
- Dataset loading time: 6.157 seconds

6.7.2 Model Complexity

- Total trainable parameters: 50,874
- MACs per forward pass: 308,688
- FLOPs per forward pass: 617,376

CNN Complexity Analysis (32×32 Input)

Architecture

Input: $1 \times 32 \times 32$

Conv1: $1 \rightarrow 4, k = 3$

Output: $4 \times 30 \times 30$

Conv2: $4 \rightarrow 8, k = 3$

Output: $8 \times 28 \times 28$

MaxPool: $2 \times 2, s = 2$

Output: $8 \times 14 \times 14$

Flatten: $8 \times 14 \times 14 = 1568$

FC1: $1568 \rightarrow 32$

FC2: $32 \rightarrow 10$

Trainable Parameters

Conv1

$$(4 \times 1 \times 3 \times 3) + 4 = 40$$

Conv2

$$(8 \times 4 \times 3 \times 3) + 8 = 296$$

FC1

$$(1568 \times 32) + 32 = 50,208$$

FC2

$$(32 \times 10) + 10 = 330$$

Total Parameters

$$40 + 296 + 50,208 + 330 = \boxed{50,874}$$

MACs (Forward Pass)

Conv1

$$4 \times 30 \times 30 \times (1 \times 3 \times 3) = 32,400$$

Conv2

$$8 \times 28 \times 28 \times (4 \times 3 \times 3) = 225,792$$

FC1

$$1568 \times 32 = 50,176$$

FC2

$$32 \times 10 = 320$$

Total MACs

$$32,400 + 225,792 + 50,176 + 320 = \boxed{308,688}$$

FLOPs

$$\text{FLOPs} = 2 \times \text{MACs}$$

$$2 \times 308,688 = \boxed{617,376}$$

6.7.3 Training Loss Curve

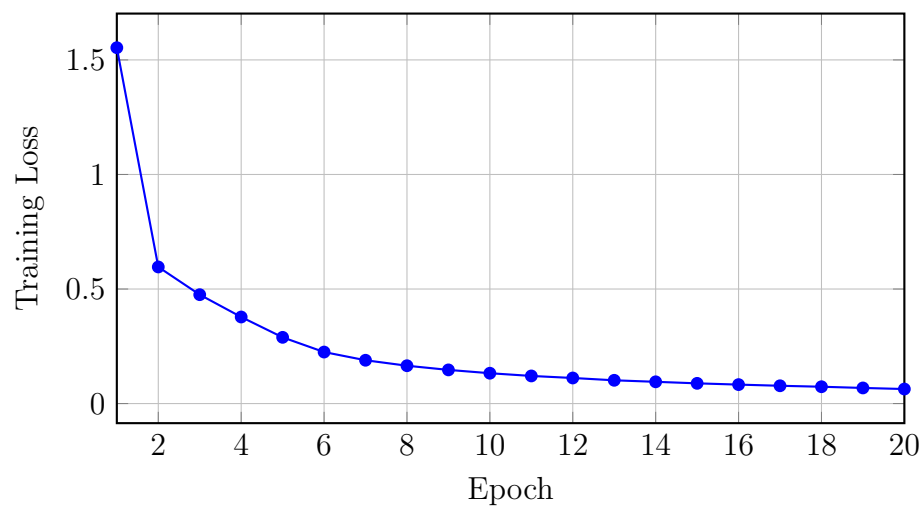


Figure 1: Training loss across epochs on MNIST dataset.

6.7.4 Accuracy Curves

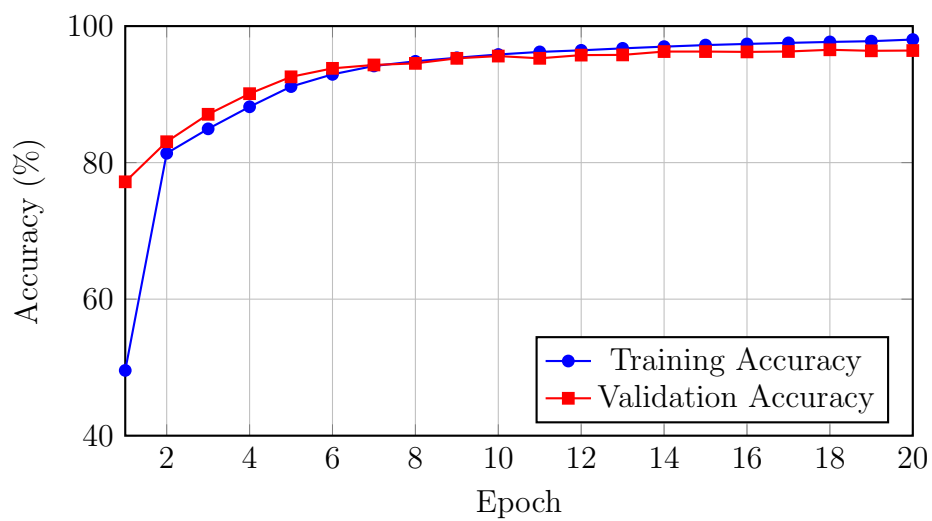


Figure 2: Training and validation accuracy across epochs.

6.7.5 Epoch Time Analysis

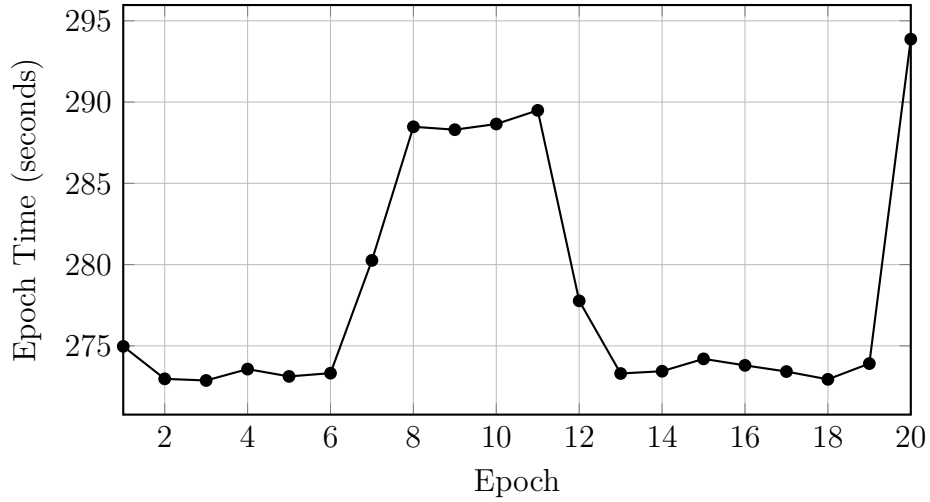


Figure 3: Per-epoch training time.

6.7.6 Metrics

Performance

- **Final Training Accuracy: 98.03%**
- **Final Validation Accuracy: 96.42%**

Observed Loading Time (MNIST Dataset)

Dataset Loading Time ≈ 6.16 seconds

Since loading occurs only once, this overhead does not increase with the number of epochs.

Total Training Time

Total Training Time ≈ 1.55 hours($5568.81seconds$)

Average Epoch Time

Average Epoch Time ≈ 4.63 minutes($278.13seconds$)

Average Throughput

Average Throughput ≈ 173 samples/sec

6.7.7 Model Complexity vs Performance

Despite having fewer than 51K parameters and less than 0.7M FLOPs per forward pass, the model achieves over 96% validation accuracy. This demonstrates an efficient trade-off between computational cost and predictive performance.

6.8 Experimental Results on Dataset 2 (CIFAR-100)

6.8.1 Dataset Statistics

- Total samples: 50,000
- Training samples: 40,000
- Validation samples: 10,000
- Input channels: 3 (RGB)
- Number of classes: 100
- Dataset loading time: 13.16 seconds

6.8.2 Model Complexity

- Total Trainable Parameters: 53916
- MACs per Forward Pass: 376368
- FLOPs per Forward Pass: 752736

6.8.3 Training Loss Curve

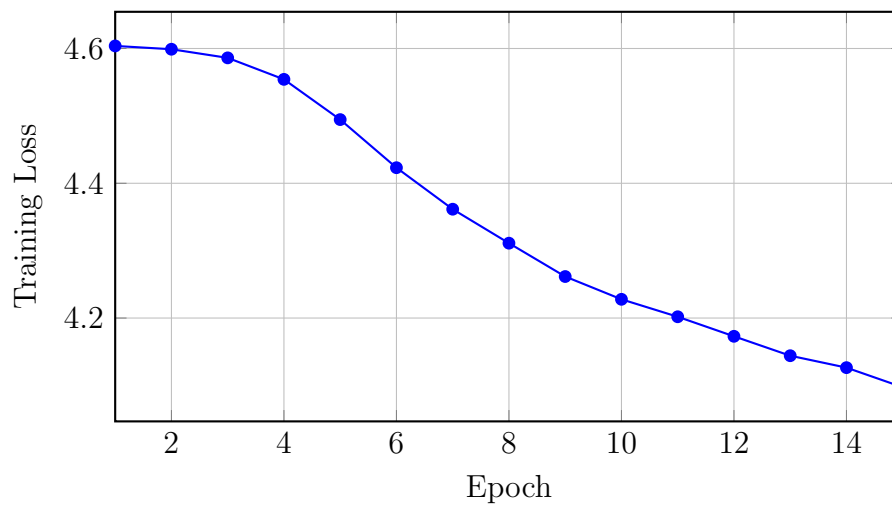


Figure 4: Training loss across epochs on Dataset 2.

6.8.4 Accuracy Curves

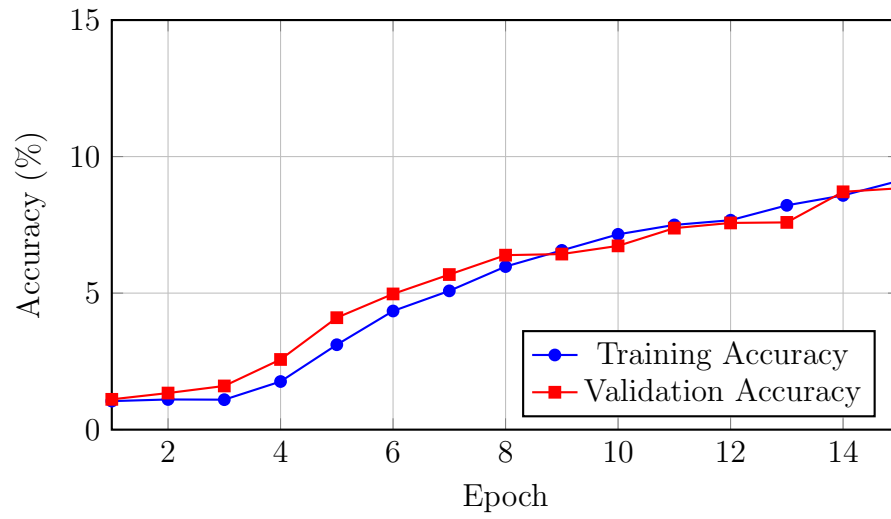


Figure 5: Training and validation accuracy across epochs.

6.8.5 Epoch Time Analysis

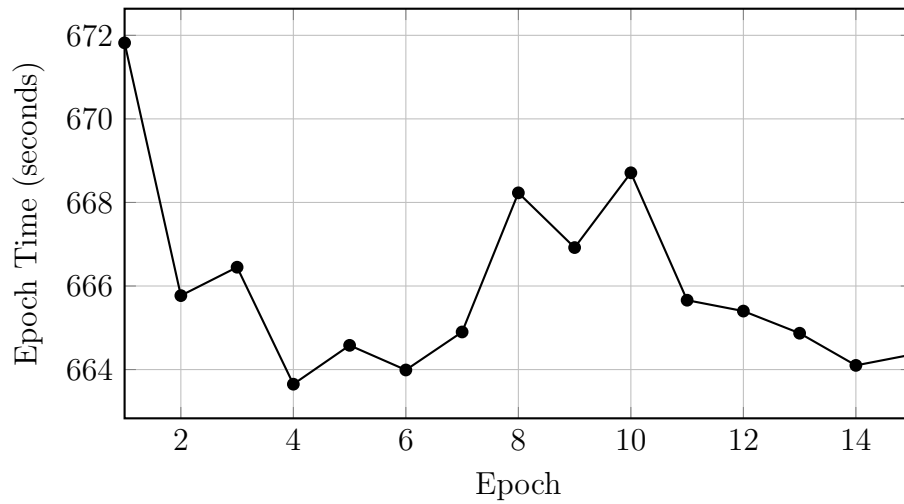


Figure 6: Per-epoch training time.

Performance (Dataset 2)

- **Final Training Accuracy: 9.12%**
- **Final Validation Accuracy: 8.84%**

Observed Loading Time (Dataset 2)

Dataset Loading Time $\approx$ 13.16 seconds

Total Training Time

Total Training Time $\approx$ 2.78 hours(10002.58 seconds)

Average Epoch Time

Average Epoch Time $\approx$ 11 minutes(665.96 seconds)

Average Throughput

Average Throughput $\approx$ 60 samples/sec

6.8.6 Model Complexity vs Performance

Having only 53,916 parameters and 0.75 MFLOPs per forward pass, the model achieves 8.84% validation accuracy on a 100-class dataset. This highlights the increased difficulty of the task and the limited capacity of the shallow architecture for large-scale multi-class classification.

A lightweight architecture was selected to satisfy the assignment constraint of completing training within 3 hours. While deeper models achieved higher validation accuracy, they exceeded the allowed training time. The final submitted architecture provides a balanced trade-off between computational efficiency and performance.

6.9 Data Loading Efficiency

Efficient dataset loading is essential to ensure that disk I/O does not dominate overall training time. The dataset is loaded once at the beginning of training using a custom `ImageFolderDataset` class, which:

- Traverses the directory structure only once
- Reads images using OpenCV
- Resizes them to 32×32
- Converts them into custom Tensor objects
- Stores all samples in memory

This design ensures that subsequent epochs reuse already loaded tensors and do not repeatedly access disk storage.

Design Decisions

- Dataset is loaded only once and reused across epochs.
- Labels are inferred from folder names and cached.
- Batching is performed via shuffled indices rather than copying data.
- Tensor data is stored in contiguous `std::vector<float>` for cache-friendly access.

7 Failed Design Decisions and Problems Faced

7.1 Memory Efficiency

Memory management was one of the most challenging aspects of implementing a custom autograd system.

Initial Problem: Out-Of-Memory (OOM) Issue During early experimentation, the training process was repeatedly terminated by the operating system after approximately 8 epochs with the message:

```
zsh: killed
```

This indicated an Out-Of-Memory (OOM) failure.

Root Cause The computational graph was being retained across iterations due to shared pointer references inside the autograd system. As a result:

- Intermediate tensors were not released
- The graph grew with every batch
- Memory consumption increased continuously

Since each batch created new Tensor objects with parent references, failing to properly release them caused exponential graph accumulation.

Fix Implemented To resolve the OOM issue, the following measures were implemented:

- Explicit deletion of intermediate tensors:

```
del x
del outputs
del loss
```

- Manual garbage collection using:

```
gc.collect()
```

- Ensured no references to past computational graphs were retained.
- Confirmed that model parameters were the only persistent tensors.

Result After these fixes:

- Memory usage stabilized across epochs
- Training successfully completed for all epochs
- No further OOM termination occurred

7.2 Improper Weight Initialization

Initially, weights were initialized using a small uniform distribution:

$$W \sim \mathcal{U}(-0.01, 0.01)$$

Observed Issues

- Activations were extremely small
- Gradients were near zero
- Training stagnated
- Validation accuracy plateaued around 60–65%

Root Cause Small initial weights resulted in weak signal propagation and ineffective gradient flow, particularly through ReLU layers.

Solution Switched to He initialization:

$$W \sim \mathcal{U}(-1, 1) \times \sqrt{\frac{2}{\text{fan_in}}}$$

This significantly improved gradient propagation and training stability.

7.3 Incorrect Bias Broadcasting

The initial implementation of bias addition failed to correctly sum gradients across the batch dimension.

This led to incorrect gradient updates and unstable training.

The fix required explicitly summing gradients over batch samples:

$$\frac{\partial L}{\partial b_j} = \sum_{i=1}^m \frac{\partial L}{\partial y_{ij}}$$

7.4 Missing parameters() Implementations

The model initially failed to load in Python due to missing method implementations in C++:

```
symbol not found: Conv2D::parameters()
```

Implementing the method resolved the binding error.

7.5 Development Environment

Experiments were done on Apple Clang 17 (ARM64) and CMake 4.2.3

A virtual environment is created with the following

- Python Version: 3.12.12
- pybind11 Version: 3.0.1
- OpenCV Version: 4.13.0

All components were verified prior to compilation and training to ensure compatibility between the Python frontend and the C++ backend.

8 Conclusion

In this assignment, a custom deep learning framework was designed and implemented with a C++ backend and Python interface. The framework was evaluated on two datasets of varying complexity. On Dataset 1 (10 classes, grayscale), the model achieved 96.42% validation accuracy with only 50,874 parameters and 0.62 MFLOPs per forward pass, demonstrating strong computational efficiency and effective gradient-based optimization.

On Dataset 2 (100 classes, RGB), the same shallow architecture achieved 8.84% validation accuracy. While performance is modest due to increased task difficulty and limited model capacity, the training remained stable and computationally efficient, confirming correctness of the implementation across different input dimensions and class counts.

The experiments validate:

- Correct forward and backward propagation
- Stable numerical behavior during training
- Accurate parameter and complexity accounting
- Efficient runtime performance within computational constraints

Sources Used

The following resources were referred to during the development of this assignment:

1. Official Documentation

- Python Documentation: <https://docs.python.org/3/>
- C++ Reference: <https://en.cppreference.com/>
- pybind11 Documentation: <https://pybind11.readthedocs.io/>
- CMake Documentation: <https://cmake.org/documentation/>
- OpenCV Documentation: <https://docs.opencv.org/>

2. General Programming References

- Stack Overflow (for debugging and build issues)
- Git Documentation

3. Course Material

- Lecture slides and materials provided in GNR 638

4. AI Assistance

- ChatGPT (OpenAI) — used for debugging support, conceptual clarification, and documentation refinement.